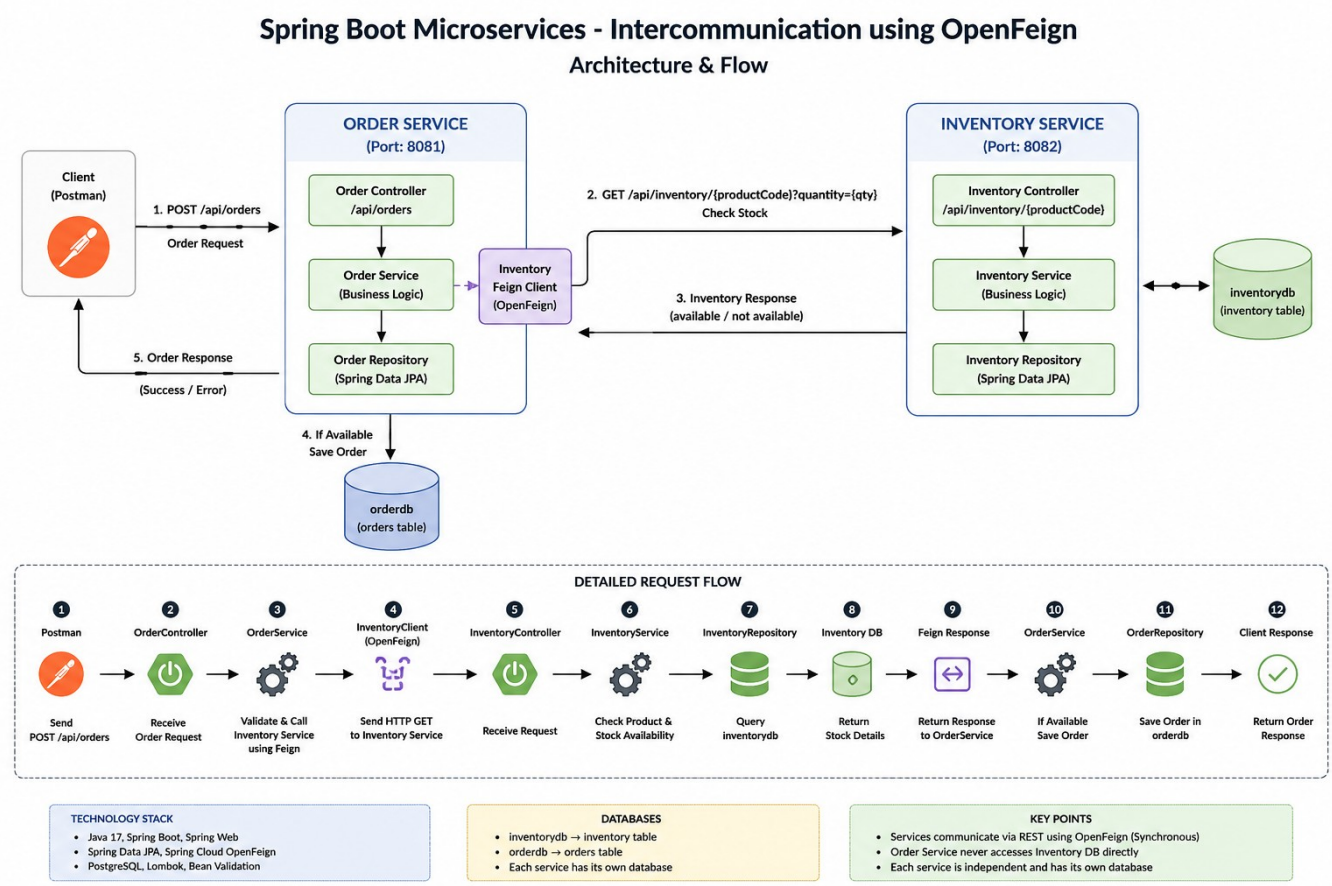
Spring Boot Microservices Intercommunication Using OpenFeign

Student Guide • Order Service + Inventory Service • PostgreSQL



Architecture overview: two independently deployable services communicating synchronously through REST/OpenFeign.

1. Learning Objectives

By the end of this lesson, students should be able to explain and implement synchronous communication between two independent Spring Boot microservices.

- Explain why each microservice owns its own database.
- Create Order Service and Inventory Service as two separate Maven projects.
- Expose an Inventory REST API for product and stock availability.
- Use Spring Cloud OpenFeign in Order Service to call Inventory Service.
- Understand the complete request/response flow and common failure scenarios.
- Test the communication using Postman and PostgreSQL.

2. Business Use Case

We are building a simplified ordering system. A customer submits an order for a product. Before Order Service accepts the order, it must ask Inventory Service whether that product exists and whether enough stock is available.

```
Customer -> Order Service -> Inventory Service -> Inventory DB
                  |
                  +-> if available -> save Order -> Order DB
                  +-> if unavailable -> return error
```

3. Why Two Separate Microservices?

Order and inventory represent different business capabilities. Keeping them separate allows each service to evolve, deploy, scale, and own its data independently.

Area	Order Service	Inventory Service
Responsibility	Create/manage orders	Maintain product stock
Port	8081	8082
Database	orderdb	inventorydb
Table	orders	inventory
Calls another service?	Yes, Inventory via Feign	No for this example

4. Core Microservice Principle: Database per Service

Order Service must never directly query the Inventory database. It asks Inventory Service through its published API. Likewise, Inventory Service should not query the Order database.

- Correct: Order Service -> HTTP/Feign -> Inventory Service -> inventorydb
- Incorrect: Order Service -> direct SQL/JPA -> inventorydb

This boundary prevents tight coupling between services and allows the Inventory team to change its internal database implementation without forcing Order Service to change, provided the API contract remains compatible.

5. What Is Microservice Intercommunication?

Microservice intercommunication is the mechanism by which independently running services exchange data or commands. Communication can be synchronous or asynchronous.

Type	How it works	Examples
Synchronous	Caller waits for the response	REST, OpenFeign, HTTP clients
Asynchronous	Caller publishes a message/event and continues	Kafka, RabbitMQ

This lesson uses synchronous REST communication: Order Service waits for Inventory Service to respond before deciding whether to save the order.

6. What Is OpenFeign?

OpenFeign is a declarative HTTP client. Instead of manually constructing an HTTP request, you define a Java interface describing the remote endpoint. Spring creates a proxy implementation and performs the HTTP call when the interface method is invoked.

```
@FeignClient(name = "inventory-service",
        url = "${inventory.service.url}")
public interface InventoryClient {

   @GetMapping("/api/inventory/{productCode}")
   InventoryResponse checkStock(
      @PathVariable("productCode") String productCode,
      @RequestParam("quantity") Integer quantity);
}
```

To OrderService, checkStock(...) looks like a Java method call. At runtime it becomes an HTTP GET request to Inventory Service.

7. Project 1 - Inventory Service

Inventory Service is an independent Spring Boot Maven project. It owns inventorydb and exposes product-stock information.

7.1 Suggested Package Structure

```
com.javacodeex.inventory
├── InventoryServiceApplication
├── controller
│   └── InventoryController
├── service
```

```
│   └── InventoryService
├── repository
│   └── InventoryRepository
├── entity
│   └── Inventory
├── dto
│   └── InventoryResponse
└── exception
    ├── ProductNotFoundException
    └── GlobalExceptionHandler
```

7.2 Inventory Entity

```
@Entity
@Table(name = "inventory")
public class Inventory {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String productCode;

    @Column(nullable = false)
    private String productName;

    @Column(nullable = false)
    private Integer quantity;

    // constructors, getters and setters
}
```

7.3 Repository

```
public interface InventoryRepository
        extends JpaRepository<Inventory, Long> {

    Optional<Inventory> findByProductCode(String productCode);
}
```

7.4 Inventory Response DTO

```
public record InventoryResponse(
    String productCode,
    String productName,
    Integer availableQuantity,
    boolean available
) {}
```

7.5 Service Logic

```
public InventoryResponse checkStock(
        String productCode,
        Integer requestedQuantity) {

    Inventory inventory = inventoryRepository
        .findByProductCode(productCode)
        .orElseThrow(() ->
            new ProductNotFoundException("Product not found: " + productCode));

    boolean available =
        inventory.getQuantity() >= requestedQuantity;

    return new InventoryResponse(
        inventory.getProductCode(),
        inventory.getProductName(),
        inventory.getQuantity(),
        available);
}
```

7.6 Inventory API

```
GET /api/inventory/{productCode}?quantity={quantity}

Example:
GET http://localhost:8082/api/inventory/P1001?quantity=2
```

```
{
  "productCode": "P1001",
  "productName": "Laptop",
  "availableQuantity": 10,
  "available": true
}
```

7.7 Inventory Configuration

```
spring.application.name=inventory-service
server.port=8082

spring.datasource.url=jdbc:postgresql://localhost:5432/inventorydb
spring.datasource.username=postgres
spring.datasource.password=postgres

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

With ddl-auto=update and a correctly scanned @Entity, Hibernate can create/update the inventory table. The PostgreSQL database inventorydb itself should already exist.

8. Project 2 - Order Service

Order Service is a second independent Spring Boot Maven project. It owns orderdb and contains the OpenFeign client.

8.1 Suggested Package Structure

```
com.javacodeex.order
├── OrderServiceApplication
├── controller
│   └── OrderController
├── service
│   └── OrderService
├── repository
│   └── OrderRepository
├── entity
│   └── Order
├── dto
│   ├── OrderRequest
│   ├── OrderResponse
│   └── InventoryResponse
├── client
│   └── InventoryClient
└── exception
    ├── InsufficientStockException
    ├── InventoryServiceUnavailableException
    └── GlobalExceptionHandler
```

8.2 Order Entity

```
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productCode;
    private Integer quantity;
    private BigDecimal price;
    private String status;
    private LocalDateTime createdAt;

    // constructors, getters and setters
}
```

8.3 Order Request DTO

```
public record OrderRequest(
    @NotBlank String productCode,
    @NotNull @Positive Integer quantity,
    @NotNull @Positive BigDecimal price
) {}
```

8.4 Enable OpenFeign

```
@SpringBootApplication
@EnableFeignClients
public class OrderServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

8.5 Inventory Feign Client

```
@FeignClient(
    name = "inventory-service",
    url = "${inventory.service.url}"
)
public interface InventoryClient {

    @GetMapping("/api/inventory/{productCode}")
    InventoryResponse checkStock(
        @PathVariable("productCode") String productCode,
        @RequestParam("quantity") Integer quantity);
}
```

8.6 Order Business Logic

```
public OrderResponse placeOrder(OrderRequest request) {

    InventoryResponse inventory =
        inventoryClient.checkStock(
            request.productCode(),
            request.quantity());

    if (!inventory.available()) {
        throw new InsufficientStockException(
            "Insufficient stock for product: "
            + request.productCode());
    }

    Order order = new Order();
    order.setProductCode(request.productCode());
    order.setQuantity(request.quantity());
```

```
    order.setPrice(request.price());
    order.setStatus("CONFIRMED");
    order.setCreatedAt(LocalDateTime.now());

    Order saved = orderRepository.save(order);

    return mapToResponse(saved);
}
```

8.7 Order API

```
POST http://localhost:8081/api/orders

{
  "productCode": "P1001",
  "quantity": 2,
  "price": 75000
}
```

8.8 Order Configuration

```
spring.application.name=order-service
server.port=8081

spring.datasource.url=jdbc:postgresql://localhost:5432/orderdb
spring.datasource.username=postgres
spring.datasource.password=postgres

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

inventory.service.url=http://localhost:8082
```

9. Complete Request Flow

1. Client sends POST /api/orders to Order Service.

2. OrderController validates and forwards the request to OrderService.

3. OrderService calls InventoryClient.checkStock(productCode, quantity).

4. Feign converts the Java method invocation into an HTTP GET request.

5. InventoryController receives the request on port 8082.

6. InventoryService applies the stock-check business logic.

7. InventoryRepository queries inventorydb using productCode.

8. Inventory Service returns product and availability information.

9. Feign converts the HTTP response into InventoryResponse in Order Service.

10. OrderService checks the available flag.

11. If available, OrderRepository saves the order in orderdb with CONFIRMED status.

12. Order Service returns the created order response to the client.

10. Success and Failure Scenarios

Scenario	Expected behavior	Order saved?
Product exists, enough stock	Inventory returns available=true; status CONFIRMED	Yes
Product exists, insufficient stock	Throw InsufficientStockException	No
Product does not exist	Inventory returns/throws product-not-found error	No
Inventory Service is down	Feign call fails; return service-unavailable response	No
Invalid order quantity/price	Bean Validation rejects request	No

11. Why Not Directly Access Inventory DB?

- It breaks service ownership and creates tight coupling.
- Order Service becomes dependent on Inventory table structure.
- A schema change in Inventory can unexpectedly break Order Service.
- Independent deployment becomes harder.
- Business rules may be bypassed if another service reads/writes the database directly.

12. Feign vs RestTemplate - Conceptual Comparison

Area	OpenFeign	Manual HTTP client approach
Programming style	Declarative interface	Explicit request-building code
Boilerplate	Lower	Usually higher
Endpoint mapping	Spring MVC annotations on interface	Construct request in code
Readability	Often easy for service-to-service clients	Depends on implementation

13. Important Limitation: Stock Check Is Not Stock Reservation

The basic example checks stock and then saves the order, but it does not reserve or deduct stock. In a real system, two orders could check the same remaining stock at nearly the same time. This is a concurrency and distributed consistency problem.

A production design may use an inventory reservation operation, optimistic/pessimistic locking, idempotency, distributed workflows such as Saga, or asynchronous events depending on the business requirement. For a beginner lesson, keep the first version focused on communication and then introduce these topics as enhancements.

14. What Happens If Inventory Service Is Down?

Because the communication is synchronous, Order Service depends on Inventory Service responding. If Inventory Service is unavailable or too slow, the order request may fail or wait until a timeout.

- Configure connection and read timeouts.
- Map Feign/network failures to a meaningful 503 response.
- Consider retries only for operations where retrying is safe.
- Use a circuit breaker such as Resilience4j for resilient production designs.
- Add monitoring, tracing, and correlation IDs for troubleshooting.

15. Testing the Use Case

15.1 Database Setup

```
CREATE DATABASE inventorydb;
CREATE DATABASE orderdb;
```

15.2 Sample Inventory Data

```
INSERT INTO inventory(product_code, product_name, quantity)
VALUES ('P1001', 'Laptop', 10);

INSERT INTO inventory(product_code, product_name, quantity)
VALUES ('P1002', 'Mobile', 20);

INSERT INTO inventory(product_code, product_name, quantity)
VALUES ('P1003', 'Headphones', 5);
```

15.3 Startup Order

- Start PostgreSQL.
- Start inventory-service on port 8082.
- Call the Inventory API directly and confirm stock is returned.

- Start order-service on port 8081.
- Call POST /api/orders.
- Observe Order Service and Inventory Service logs.
- Verify the confirmed order in orderdb.

16. Classroom Demo Scenarios

- Happy path: P1001, quantity 2 when stock is 10.
- Insufficient stock: request quantity 20 when stock is 10.
- Unknown product: request P9999.
- Stop Inventory Service and call Order Service to demonstrate dependency failure.
- Restart Inventory Service and retry the request.
- Show that Order Service contains no InventoryRepository and no inventorydb connection.

17. Key Annotations to Remember

Annotation	Purpose
@FeignClient	Declares an HTTP client for another service.
@EnableFeignClients	Enables discovery/registration of Feign client interfaces.
@GetMapping	Maps the remote Inventory GET endpoint.
@PathVariable	Supplies productCode in the URL path.
@RequestParam	Supplies requested quantity as a query parameter.
@Entity	Marks a class as a JPA entity.
@RestController	Exposes REST endpoints.
@Service	Contains business logic.
@RestControllerAdvice	Centralizes REST exception handling.

18. Interview / Viva Questions

- What is microservice intercommunication?
- What is the difference between synchronous and asynchronous communication?
- Why does Order Service not access Inventory DB directly?
- What problem does OpenFeign solve?
- What does @EnableFeignClients do?
- How does a Feign interface become an HTTP request?
- What happens when Inventory Service returns available=false?
- What happens when Inventory Service is unavailable?
- Why should each service own its own database?
- Is checking stock enough to guarantee stock will still be available when the order is saved? Why not?

19. Student Exercise

Extend the application after the basic communication works:

- Add GET /api/orders/{id}.
- Add an endpoint to create/update inventory.
- Add stock deduction or reservation after successful order placement.
- Return proper 400, 404, 409, and 503 error responses.
- Add Feign timeout configuration.
- Add Resilience4j circuit breaker.
- Replace the fixed Inventory URL with service discovery in an advanced version.
- Add unit tests and integration tests for both services.

20. Final Summary

In this architecture, Order Service and Inventory Service are two independent Spring Boot applications with separate databases. Order Service owns order data; Inventory Service owns stock data. When an order arrives, Order Service uses an OpenFeign client to synchronously call Inventory Service. Inventory Service checks its own database and returns availability. Order Service saves the order only when the stock condition is satisfied.

The central lesson is not only how to use Feign, but also how to preserve microservice boundaries: communicate through APIs, keep data ownership local to each service, handle remote failures, and avoid treating a distributed system like a single monolithic database.